redrob | H2S

INDIA.RUNS

Build what next India runs on

Team Name: IntelliRank

Team Leader Name: Muskan Sundrani

Problem Statement:

Build an AI system that ranks candidates for a Senior AI Engineer role by true role fit, not keyword count.

Solution Overview

What is your proposed solution?

- IntelliRank is a hybrid candidate ranking system combining structured profile scoring (title fit, trusted skill evidence, career quality, experience, location, education), semantic relevance (TF-IDF similarity to JD intent), behavioral readiness signals, and profile consistency checks.

What differentiates your approach from traditional candidate matching systems?

- Traditional systems over-rely on keyword overlap
- IntelliRank uses role-fit reasoning: hard penalties for title mismatch traps (e.g., non-ML titles with stuffed AI keywords)
- Skill trust scoring from duration + endorsements + proficiency
- Career narrative quality and production ML evidence, plus behavioral availability (response rate, notice period, open-to-work)

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- Role: Senior AI Engineer (Founding Team)
- Experience sweet spot: ~5–9 years
- Core capability: retrieval, embeddings, vector search, ranking/evaluation
- Practical orientation: product-company ML production > pure research
- Location preference: India (Noida/Pune preferred)
- Recruitability: active candidates, better response behavior, shorter notice period

Most important candidate signals

- Current title and title history relevance
- Trusted core skills: embeddings, FAISS/Pinecone/Qdrant/pgvector, semantic search, ranking
- Career evidence: shipped/deployed/production systems
- Platform behavior: response rate, activity recency, open-to-work, notice period
- Consistency checks: suspicious timelines, unrealistic expert claims

Ranking Methodology

How does the system retrieve, score, and rank?

- Load candidate profiles from JSONL
- Build profile text; compute semantic similarity vs. JD vector (precomputed TF-IDF)
- Extract structured features per candidate
- Compute weighted base score
- Apply behavioral and honeypot multipliers
- Sort by score (rounded), then candidate_id for tie-break
- Return top-100 with reasoning

Models, algorithms & heuristics

- TF-IDF (scikit-learn) for semantic relevance
- Weighted scoring engine for interpretable hybrid ranking
- Heuristic gates/penalties for title traps & suspicious profiles

Signal Combination

```
Final Score =
BaseWeightedScore ×
BehavioralMultiplier ×
HoneypotMultiplier
```

Explainability & Data Validation

How are ranking decisions explained?

- Title + years of experience
- Top matching retrieval/embedding/ranking skills
- Career quality context
- Behavioral strengths/concerns

Preventing hallucinations

- Reasoning generated from computed features only
- No external LLM inference used at ranking-time
- Every statement maps to profile fields or derived feature outputs

Handling low-quality / suspicious profiles

- Honeypot multiplier penalizes expert skills with 0 months
- Flags timeline inconsistencies
- Flags non-ML titles with heavy AI keyword stuffing
- Title hard-cap prevents fake top ranking from keyword stuffing alone

End-to-End Workflow

1. Input JD requirements (structured constants + JD narrative)
2. Offline precompute step (precompute.py) builds TF-IDF artifacts
3. Online ranking step (rank.py) processes all candidates on CPU
4. Features + semantic signals → hybrid scoring
5. Top 100 ranked candidates generated
6. Submission file exported: candidate_id, rank, score, reasoning
7. Validation via validate_submission.py

System Architecture

Data Layer

- data/candidates.jsonl
- artifacts/ — TF-IDF vectors + candidate IDs

Feature Layer

- src/features.py
- title, skills, experience, location, education, behavioral, career signals

Intelligence Layer

- src/scorer.py — hybrid weighted score + multipliers
- src/honeypot.py — suspicious profile penalties
- src/reasoning.py — human-readable explanations

Execution Layer

- precompute.py — offline vector prep
- rank.py — final ranking + CSV output
- validate_submission.py — spec compliance

Results & Performance

Ranking quality highlights

- Top candidates are consistently ML/AI-focused titles
- Strong alignment to JD-required retrieval/embedding/vector skills
- Behavioral readiness and notice-period concerns are surfaced in reasoning
- Trap profiles are effectively pushed down by title gate + trust penalties

Runtime & compute constraints

~2 min	~15 sec	Off
Precompute (offline, CPU)	Ranking for 100K candidates (CPU only)	Network during ranking

Technologies Used

Python 3.9

Fast, reliable ML scripting ecosystem

NumPy / SciPy

Efficient numeric and sparse operations

scikit-learn

TF-IDF vectorization and sparse similarity

Pandas

Result shaping and quick analysis

Streamlit / Gradio

Sandbox/demo interfaces

Git + Streamlit Cloud

Reproducible hosting and submission demo

Why selected

- Lightweight, interpretable, CPU-friendly stack
- Fast iteration and reproducibility under strict runtime constraints

Submission Assets

GitHub Repository	https://github.com/muskan-sundrani-1707/redrob-hackathon
Sandbox Link (Streamlit)	https://redrob-hackathon-uqe4tmxbuuglqf9z96enr6.streamlit.app
Ranked Output File	`submission.csv`
Approach Deck PDF	`IntelliRank_Redrob_Deck.pdf`
Metadata	`submission_metadata.yaml`

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU

IntelliRank improves recruiter trust by ranking candidates on real role fit, not surface-level keyword overlap.

Next Improvements

- Learning-to-rank with labeled relevance tiers
- Cross-encoder re-ranking for top-N refinement
- Richer longitudinal career trajectory modeling

Muskan Sundrani — IntelliRank — Redrob Data & AI Challenge